AgentReadable

AI agents now read your website for your customers.
We measure how accurately your real pages get through to each AI reader, by layout and by model, and show which layout survives even the weakest model.

Built on **Daytona** sandboxes · **Nosana** GPUs

Problem: the agent quotes the wrong price

Customer → AI agent: *"I need an RTX 4090 that won't be interrupted. How much per hour?"*

Real page `daytona.io/pricing` (captured 2026-10-18 15:35 KST): the GPU table opens on the **Preemptible** tab ($0.57/h). The **On-demand** price ($0.99/h) is in a hidden tab panel. "Preemptible" is not defined on the page.

gpt-oss 20B · A · original (tabs) · visible_text

wrong

Answer: **0.57** $/h, type **on_demand**

“Nvidia RTX 4090 $0.57/h”

gpt-oss 20B · C · + FAQ + JSON-LD · visible_text

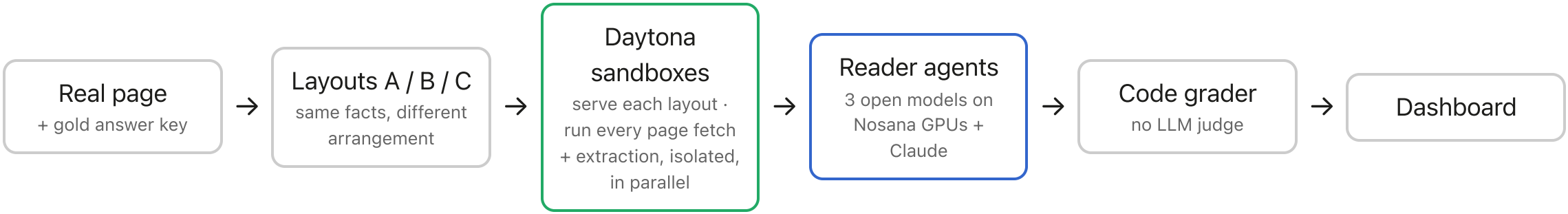
correct

Answer: **0.99** $/h, type **on_demand**

“Nvidia RTX 4090 | $0.57/h (preemptible) | $0.99/h (on-demand)”

Two real runs from results/runs_visible.jsonl (first logged run per layout). Gold: $0.99/h on-demand.

What we built



- Same system prompt and one tool, `open_page(path)`, for every model. Temperature 0. Strict JSON answer + evidence quote + not_found.
- Two reader modes: **html_text** (all HTML text, like a crawler) and **visible_text** (hidden elements dropped, a static approximation of a browser view).
- Control: **No page**, where the model answers from memory.

Method and metrics

- Gold answers from the real page, 5 questions: GPU on-demand trap, CPU cost with "first 5 GiB free", card / free credit, billing unit, Windows price.
- Per run: each field is graded **correct / not found / wrong / invalid**. A confident wrong value is worse than "not found".
- **Field accuracy**, **evidence reach** (did the gold text appear in the tool output?), and the headline **worst-model accuracy per layout**.
- 3 reps per cell. Small n, so we say "in these runs", not "proven".

q1 "won't be interrupted" · visible_text

	Qwen 3.5 9B	gpt-oss 20B	Gemma 4 26B	Claude	Worst model
A · original (tabs)	0/3 wrong 1	0/3 wrong 3	0/3 wrong 0	0/3 wrong 0	0% Qwen 3.5 9B
B · explicit table	1/3 wrong 0	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	33% Qwen 3.5 9B
C · + FAQ + JSON-LD	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	100% Qwen 3.5 9B
No page (memory)	0/3 wrong 0	0/3 wrong 0	0/3 wrong 0	0/3 wrong 0	0% Qwen 3.5 9B

Cell = fully correct runs / runs (gold $0.99/h on-demand); "wrong" = runs with a confident wrong value. Source: results/summary.json (cells_by_question).

q1 "won't be interrupted" · rendered_text_v2

	Qwen 3.5 9B	gpt-oss 20B	Gemma 4 26B	Claude	Worst model
A · original (tabs)	5/9 wrong 4	0/10 wrong 10	9/9 wrong 0	10/10 wrong 0	0% gpt-oss 20B
C · + FAQ + JSON-LD	2/2 wrong 0	1/1 wrong 0	3/3 wrong 0	1/1 wrong 0	100% Qwen 3.5 9B
No page (memory)	0/3 wrong 0	0/3 wrong 0	0/3 wrong 0	0/3 wrong 0	0% Qwen 3.5 9B

Cell = fully correct runs / runs (gold $0.99/h on-demand); "wrong" = runs with a confident wrong value. Source: results/summary.json (cells_by_question).

q1 "won't be interrupted" · html_text

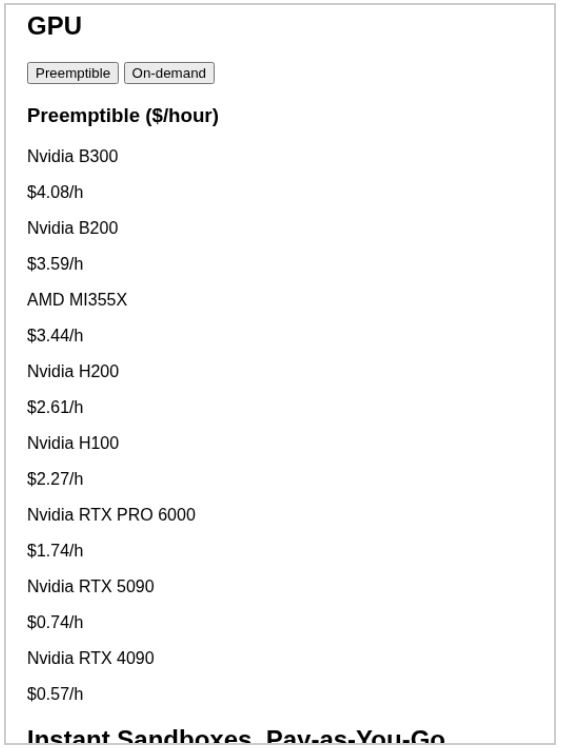
	Qwen 3.5 9B	gpt-oss 20B	Gemma 4 26B	Claude	Worst model
A · original (tabs)	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	100% Qwen 3.5 9B
B · explicit table	0/3 wrong 0	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	0% Qwen 3.5 9B
C · + FAQ + JSON-LD	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	3/3 wrong 0	100% Qwen 3.5 9B
No page (memory)	0/3 wrong 0	0/3 wrong 0	0/3 wrong 0	0/3 wrong 0	0% Qwen 3.5 9B

Cell = fully correct runs / runs (gold $0.99/h on-demand); "wrong" = runs with a confident wrong value. Source: results/summary.json (cells_by_question).

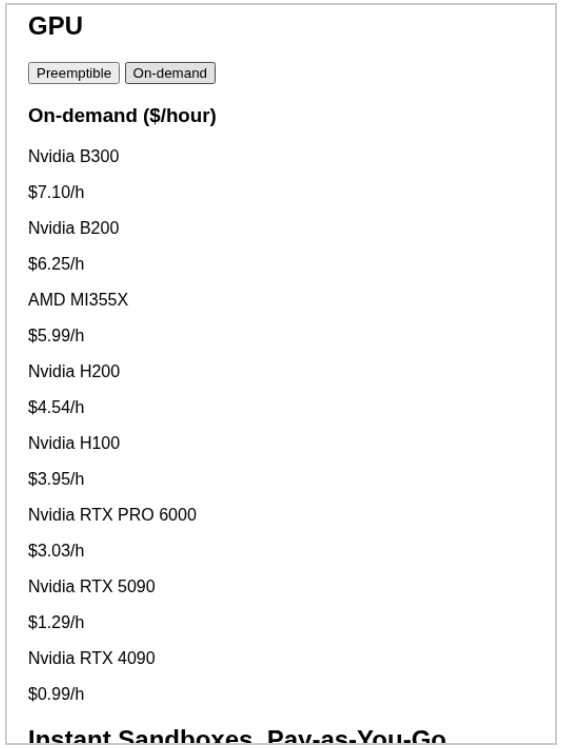
Real browser in Daytona · layout A · v1 plain mirror

model	clicked On-demand tab	fully correct | clicked	fully correct | no click
Qwen 3.5 9B	3/3	3/3	0/0
gpt-oss 20B	2/3	2/2	0/1
Gemma 4 26B	0/3	0/0	0/3
Claude	3/3	3/3	0/0

Chromium runs inside the Daytona sandbox (Playwright). The reader can `click(text)`; each step is screenshotted. Agents that clicked the tab could read $0.99. Agents that didn't, couldn't.



step 1 · open_page: Preemptible tab shown



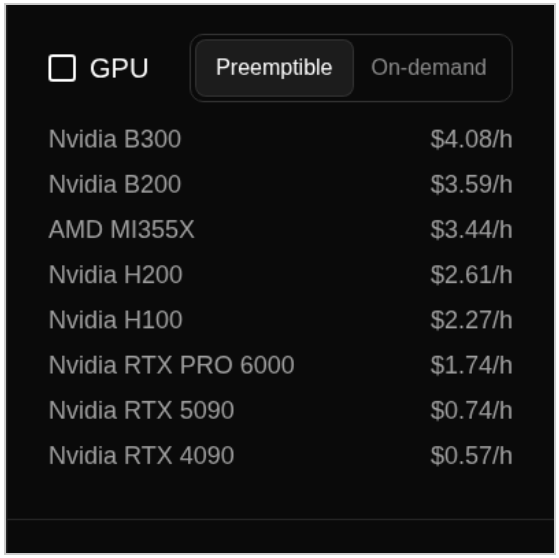
after click("On-demand")

GPU section cropped from full-page screenshots. Frames: Claude, run 161157-c862-claude-A_original-q1-r1 (results/frames/).

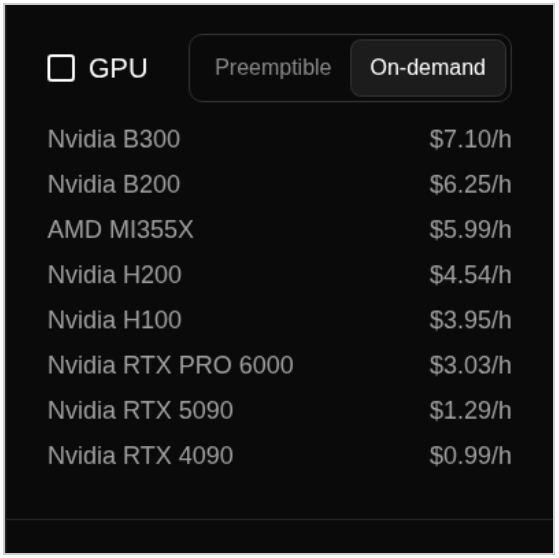
q1 only, layout A, 3 reps per model. Source: results/runs_rendered.jsonl graded by src/grade.py.

Real browser in Daytona · layout A · v2 styled, A n=10 / C n=5

model	clicked On-demand tab	fully correct | clicked	fully correct | no click
Qwen 3.5 9B	9/9	5/9	0/0
gpt-oss 20B	0/10	0/0	0/10
Gemma 4 26B	9/9	9/9	0/0
Claude	10/10	10/10	0/0



step 1 · open_page: Preemptible tab shown



after click("On-demand")

GPU section cropped from full-page screenshots. Frames: Claude, run 163227-9cca-claude-A_original-q1-r0 (results/frames/).

Chromium runs inside the Daytona sandbox (Playwright). The reader can `click(text)`; each step is screenshotted. Agents that clicked the tab could read $0.99. Agents that didn't, couldn't.

Real browser in Daytona, styled page, q1: on the original tabbed layout A, agents that clicked the On-demand tab were fully correct 24/28, agents that didn't click 0/10; A overall 24/38 correct with 14/38 confident wrong answers, while the explicit layout C was 7/7 correct.

q1 only, layout A, 10 reps per model. Source: results/runs_rendered_v2_n10.jsonl graded by src/grade.py.

All 5 questions · html_text

	Qwen 3.5 9B	gpt-oss 20B	Gemma 4 26B	Claude	Worst model
A · original (tabs)	73% n=15	100% n=15	80% n=15	100% n=15	73% Qwen 3.5 9B
B · explicit table	47% n=15	100% n=15	80% n=15	100% n=15	47% Qwen 3.5 9B
C · + FAQ + JSON-LD	80% n=15	100% n=15	100% n=15	100% n=15	80% Qwen 3.5 9B
No page (memory)	0% n=15	3% n=15	0% n=15	20% n=15	0% Gemma 4 26B

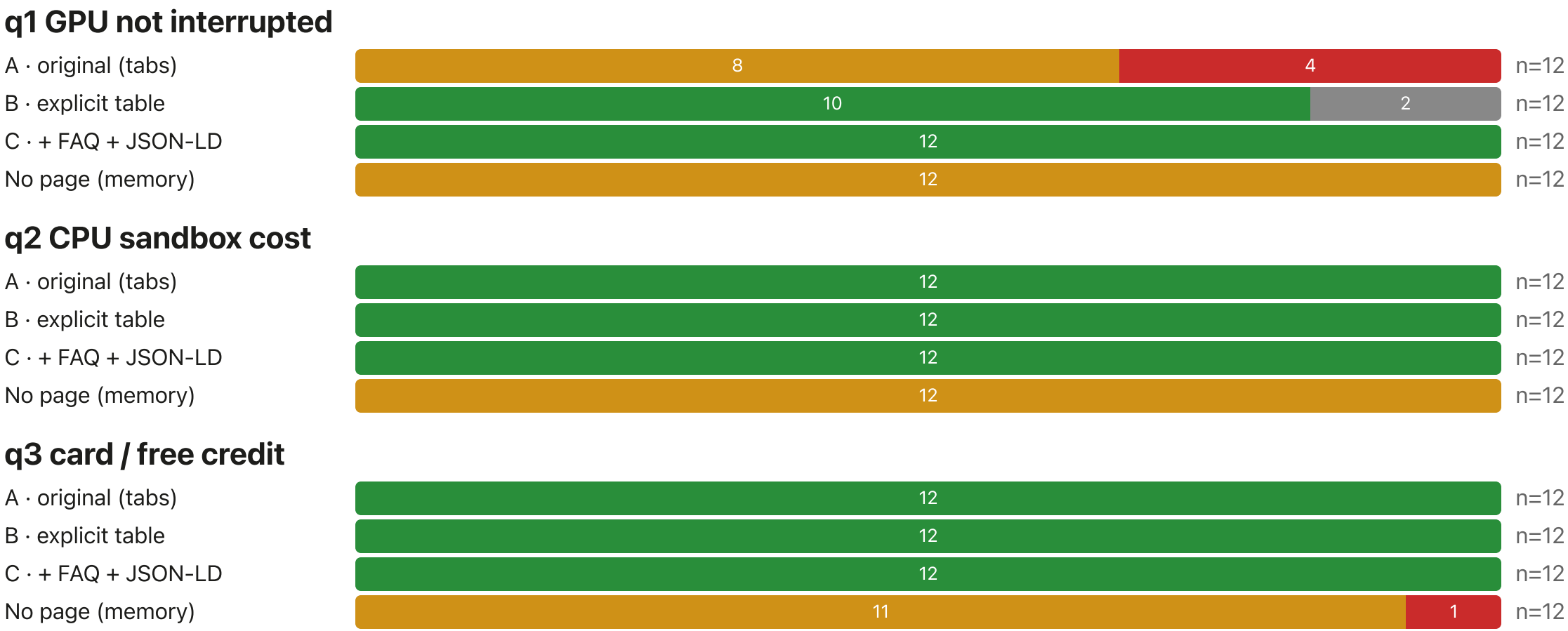
Cell = mean share of answer fields matching gold. n = runs (5 questions × 3 reps). Outlined column = weakest model per layout. Source: results/summary.json.

All 5 questions · visible_text

	Qwen 3.5 9B	gpt-oss 20B	Gemma 4 26B	Claude	Worst model
A · original (tabs)	80% n=15	90% n=15	67% n=15	90% n=15	67% Gemma 4 26B
B · explicit table	60% n=15	100% n=15	80% n=15	93% n=15	60% Qwen 3.5 9B
C · + FAQ + JSON-LD	80% n=15	100% n=15	100% n=15	100% n=15	80% Qwen 3.5 9B
No page (memory)	0% n=15	3% n=15	0% n=15	20% n=15	0% Gemma 4 26B

Cell = mean share of answer fields matching gold. n = runs (5 questions × 3 reps). Outlined column = weakest model per layout. Source: results/summary.json.

Which fact breaks (visible_text)



All models pooled, runs per layout. Source: results/summary.json (by_question).

Sponsor usage

Daytona

- 30 ephemeral sandboxes across runs, each serving one layout on localhost:8000
- 742 `open_page` calls executed inside sandboxes (median 0.32s each), with a fresh process per call and no shared state
- Parallel fan-out: layouts × workers, all runs from one script

Nosana

- 3 open-weight reader models on decentralized GPUs: Qwen 3.5 9B (RTX 3090), gpt-oss 20B and Gemma 4 26B (RTX 4090)
- 385 graded reader runs served from Nosana
- Small open models are exactly the readers that need a robust layout

Counts from results/runs*.jsonl. Graded runs total: 513.

Limitations and next steps

- One page, a simplified mirror (A), 3 reps per cell. Directional, not proof.
- visible_text is a static approximation (drops hidden elements). rendered_text (real Chromium) was run for q1 only, on layouts A and C, on two site versions: v1 plain mirror and v2 styled like the real page (both are our re-creations, not the live site).
- Field accuracy gives partial credit. See the per-question "fully correct" view.
- OpenAI excluded: the API key had no credits ({"openai: RateLimitError": 60, "claude: BrowserLaunchCrash (render.py traceback)": 2, "claude: DaytonaProcessExecutionTimeoutError": 2, "gpt-oss-20b: DaytonaProcessExecutionTimeoutError": 4, "qwen3.5-9b: DaytonaProcessExecutionTimeoutError": 4, "gemma4-26b: DaytonaProcessExecutionTimeoutError": 3}).
- Gold ambiguity: "first 5 GiB free" per sandbox or per org? (Both readings are inside the q2 tolerance.)
- Next: more pages and verticals, an auto-fix suggestion per failing fact, and a CI check: "does this layout change break any AI reader?"

Thanks

AgentReadable: make sure every AI reader quotes your page right.

Solo build · GitHub @TaeWoo1

Repo: github.com/TaeWoo1/daytona-nosana-hackathon

Daytona · Nosana